x64 系统调用约定与 SSN 提取知识点整理

- **核心目标**

理解 x64 系统调用流程（kernel32!WriteProcessMemory → ntdll!NtWriteVirtualMemory → syscall → 内核），掌握 x64 调用约定（rcx/rdx/r8/r9 放前 4 个参数，eax 放 SSN），能从 ntdll.dll 的 Nt* 函数机器码中手动提取 SSN，并用内联汇编直接触发 syscall。

- **涉及的核心类型定义**

(一) NTSTATUS（原生 API 返回类型）

```
typedef LONG NTSTATUS;        // 4 字节有符号整数
```

> **用途：**syscall 的返回值，0 表示成功（STATUS_SUCCESS）。

(二) 函数指针类型（无 IAT 版使用）

```
// NtWriteVirtualMemory 的原型（用于理解参数顺序）
typedef NTSTATUS(NTAPI* pNtWriteVirtualMemory)(
    HANDLE      ProcessHandle,          // 参数 1 → rcx
    PVOID       BaseAddress,            // 参数 2 → rdx
    PVOID       Buffer,                 // 参数 3 → r8
    SIZE_T      NumberOfBytesToWrite,   // 参数 4 → r9
    PSIZE_T     NumberOfBytesWritten    // 参数 5 → 栈
);
```

> **关键：**前 4 个参数放寄存器，第 5 个及以后放栈。这是 x64 调用约定的核心。

(三) ULONG_PTR（参数传递类型）

```
typedef ULONG_PTR;        // x64 下为 8 字节无符号整数
```

> **用途：**DoSyscall 的参数类型，保证参数宽度与寄存器匹配。

- **涉及的关键宏与常量**

宏/常量	值	作用
0x4C 8B D1 B8	—	Nt* 函数的标准开头：mov r10, rcx; mov eax, imm32
STATUS_SUCCESS	0	syscall 成功返回码
NTAPI	__stdcall	原生 API 调用约定（x64 下统一为 Microsoft x64 calling convention）

- **涉及的所有 API 函数**

➢ **明文版专用**

(一) GetModuleHandleA 获取 ntdll.dll 基址

```
HMODULE GetModuleHandleA(
  [in, optional] LPCSTR lpModuleName
);
```

[in, optional] lpModuleName

加载的模块的名称（.dll 或 .exe 文件）。如果省略文件扩展名，则会追加默认库扩展名 .dll。文件名字符串可以包含尾随点字符（.），以指示模块名称没有扩展名。名称与当前映射到调用进程的地址空间的模块的名称进行比较（大小写不敏感）。

返回值

如果函数成功，则返回值是指定模块的句柄。

如果函数失败，则返回值为 NULL。

(二) GetProcAddress 获取 NtWriteVirtualMemory 地址

```
FARPROC GetProcAddress(
  [in] HMODULE hModule,
  [in] LPCSTR  lpProcName
);
```

[in] hModule

包含函数或变量的 DLL 模块的句柄。

GetProcAddress 不会从使用 LOAD_LIBRARY_AS_DATAFILE 标志加载模块中检索地址。

[in] lpProcName

函数或变量名称，或函数的序号值。

返回值

如果函数成功，则返回值是导出的函数或变量的地址。

如果函数失败，则返回值为 NULL。要获得更多的错误信息，请调用 GetLastError。

- **系统调用核心（非 API，是 CPU 指令）**

syscall 指令

- **作用：**从用户态陷入内核，执行系统调用
- **入口：**eax = SSN（系统服务编号）
- **参数：**rcx = 参数 1，rdx = 参数 2，r8 = 参数 3，r9 = 参数 4
- **内核约定：**r10 = rcx 的备份（内核用 r10 而非 rcx）
- **返回：**eax = NTSTATUS

- **完整的利用链（SSN 提取 + syscall 调用）**

```
第 1 步：获取 ntdll.dll 基址
    明文版：GetModuleHandleA("ntdll.dll")
    无 IAT 版：GetModuleBaseByHash(NHash32::hash("ntdll.dll"))
            |
            └──── 拿到 hNtdll
第 2 步：定位 NtWriteVirtualMemory 函数地址
    明文版：GetProcAddress(hNtdll, "NtWriteVirtualMemory")
    无 IAT 版：GetProcAddressEx(hNtdll, NHash32::hash("NtWriteVirtualMemory"))
            |
            └──── 拿到 pNtWrite（函数地址）
第 3 步：提取 SSN（核心）
    检查 pNtWrite[0..3] 是否为 4C 8B D1 B8
            |
            ├──── 是 → SSN = *(DWORD*)(pNtWrite + 4)
            |
            └──── 否 → 函数被 Hook 或不是标准 Nt* 存根
第 4 步：用 syscall 调用（无 IAT 版）
    ① eax = SSN
    ② rcx = 参数 1（ProcessHandle）
    ③ rdx = 参数 2（BaseAddress）
    ④ r8  = 参数 3（Buffer）
    ⑤ r9  = 参数 4（NumberOfBytesToWrite）
    ⑥ r10 = rcx（内核约定）
    ⑦ syscall
```

⑧ 返回值在 eax 中

文字总结：

- 拿 ntdll 基址 → 明文版用 GetModuleHandleA，无 IAT 版用哈希遍历
- 定位函数 → 明文版用 GetProcAddress，无 IAT 版用 GetProcAddressEx
- 提取 SSN → 检查 4 字节特征，读取偏移 +4 的 DWORD
- 触发 syscall → 按 x64 调用约定填寄存器，执行 syscall 指令

- **x64 系统调用约定详解**

(一) 完整调用链

用户态：

```
kernel32!WriteProcessMemory
        |
        └──── 转发到 ntdll!NtWriteVirtualMemory
                  |
                  └──── 标准存根:
                        4C 8B D1          mov r10, rcx
                        B8 XX XX XX XX    mov eax, SSN
                        0F 05             syscall
                        C3                ret
        |
        ▼
```

内核态：

```
KiSystemCall64（syscall 入口）
        |
        └──── 查系统服务表（SSDT）
        |
        └──── 执行 NtWriteVirtualMemory 的内核实现
        |
        └──── 返回用户态
```

(二) 寄存器分配

寄存器	用途	说明
eax	SSN	系统服务编号
rcx	参数 1	用户态约定
rdx	参数 2	
r8	参数 3	
r9	参数 4	
r10	rcx 的备份	内核约定，syscall 会清空 rcx
栈	参数 5+	从 [rsp+0x28] 开始

(三) 为什么需要 mov r10, rcx

- syscall 指令执行时，rcx 会被硬件自动覆盖为返回地址
- 内核需要参数 1，所以用户态存根先把 rcx 备份到 r10
- 内核从 r10 读取参数 1

- **核心偏移量速查（Windows 10/11 x64）**

描述	表达式	偏移值（HEX）

Nt* 函数开头特征	funcAddr[0..3]	4C 8B D1 B8
SSN 位置	funcAddr + 4	+0x04
第 5 个参数在栈上	[rsp + 0x28]	0x28

- **其他必要的扩展知识点**

(一) 为什么 Nt* 函数开头是 4C 8B D1 B8

这是微软编译器生成的标准系统调用存根：

```
4C 8B D1          mov r10, rcx      ; 备份参数 1 到 r10
B8 XX XX XX XX    mov eax, SSN      ; SSN 放入 eax
0F 05             syscall           ; 进入内核
C3                ret               ; 返回
```

如果开头不是这 4 个字节：

- 函数被 EDR 的 Inline Hook 修改了（通常改成 jmp 到监控函数）
- 或者不是标准 Nt* 存根（某些转发函数）

(二) 直接 syscall 与调用 ntdll 函数的区别

维度	调用 ntdll!NtWriteVirtualMemory	直接 syscall
经过用户态 Hook	是（EDR 可能 Hook）	否（直接进内核）
需要 SSN	不需要（存根里有）	需要（自己填 eax）
检测难度	较低（Hook 可捕获）	较高（栈回溯异常）
兼容性	系统保证	SSN 随版本变化

(三) 内联汇编的两种写法

- **GCC 版（代码中使用）：**

```
__asm__ __volatile__ (
    "mov %[ssn], %%eax\n"
    "mov %[a1],  %%rcx\n"
    // ...
    "syscall\n"
    : [out] "=r" (status)
    : [ssn] "r" (ssn), [a1] "r" (a1), ...
    : "rax", "rcx", "rdx", "r8", "r9", "r10", "r11", "memory"
);
```

- **MSVC 版：x64 下不支持 __asm 内联汇编，必须用：**
  - 单独的 .asm 文件（MASM）
  - 或使用 __intrin.h 的 __fastcall + 汇编包装函数
  - 或使用 RtlCaptureContext + NtContinue

(四) SSN 的稳定性

- 同一 Windows 版本内：SSN 固定不变
- 不同 Windows 版本：SSN 可能变化（微软不保证跨版本兼容）
- Windows 10 各版本：NtWriteVirtualMemory 的 SSN 在 1903 之前是 0x37，之后变为 0x3A（示例，实际需提取）

结论：不能硬编码 SSN，必须运行时提取

(五) 为什么 ExtractSSN 要检查 4 个字节

```
if (funcAddr[0] != 0x4C || funcAddr[1] != 0x8B ||
   funcAddr[2] != 0xD1 || funcAddr[3] != 0xB8) {
```

```
    return 0;       // 不是标准存根
}
```

如果只检查 B8（1 字节）：

- 误判率高（很多指令以 B8 开头）
- 可能读到错误的 SSN

检查 4 字节：

- 4C 8B D1 是 mov r10, rcx 的机器码
- B8 是 mov eax, imm32 的操作码
- 4 字节组合几乎唯一标识标准 Nt* 存根

(六) 失败路径：SSN 提取返回 0 怎么办

```
if (ssn == 0) {
    std::cerr << "[] SSN 提取失败（可能被 Hook）" << std::endl;
    return 1;
}
```

SSN = 0 的含义：

- 函数开头不是标准存根（被 Hook）
- 或者 NtWriteVirtualMemory 的 SSN 恰好是 0（极少见，通常 SSN 从 0 开始，但 NtWriteVirtualMemory 不会是 0）

被 Hook 的应对：

- 从 ntdll 的磁盘文件重新映射一份干净的副本（反射加载）
- 从 kernel32!WriteProcessMemory 的转发链中找原始地址
- 使用 Hell's Gate / Halo's Gate 技术（从相邻函数推算 SSN）

(七) 无 IAT 版的 DoSyscall 参数传递

```
NTSTATUS DoSyscall(DWORD ssn, ULONG_PTR a1, ULONG_PTR a2,
                   ULONG_PTR a3, ULONG_PTR a4);
```

为什么用 ULONG_PTR 而不是具体类型：

- ULONG_PTR 在 x64 下是 8 字节，与寄存器宽度匹配
- 避免类型转换警告
- 通用性强（可以传任意指针或整数）

第 5 个参数：

- 代码中 NtWriteVirtualMemory 只用了 4 个参数
- 第 5 个参数（NumberOfBytesWritten）传 NULL 时，可以不传（或传 0）
- 如果需要第 5 个参数，必须用汇编在栈上分配空间

- **核心代码片段**

(一) 提取 SSN

```
DWORD ExtractSSN(BYTE* funcAddr) {
    if (!funcAddr) return 0;
    if (funcAddr[0] != 0x4C || funcAddr[1] != 0x8B ||
        funcAddr[2] != 0xD1 || funcAddr[3] != 0xB8) {
        return 0;
    }
    return *(DWORD*)(funcAddr + 4);
}
```

(二) 内联汇编 syscall（GCC）

```
__asm__ __volatile__ (
        "mov %[ssn], %%eax\n"
        "mov %[a1],    %%rcx\n"
        "mov %[a2],    %%rdx\n"
        "mov %[a3],    %%r8\n"
        "mov %[a4],    %%r9\n"
        "mov %%rcx,    %%r10\n"
        "syscall\n"
        "mov %%eax,    %[out]\n"
        : [out] "=r" (status)
        : [ssn] "r" (ssn),
          [a1] "r" (a1),
          [a2] "r" (a2),
          [a3] "r" (a3), [a4] "r" (a4)
        : "rax", "rcx", "rdx", "r8", "r9", "r10", "r11", "memory"
);
```

(三) 打印前 8 字节验证

```
for (int i = 0; i < 8; i++) {
        std::cout << std::hex << std::setw(2) << std::setfill('0')
                          << (int)pNtWrite[i] << " ";
}
```